Lösungen zum Leitprogramm Kompression

Ausführliche Lösungen zum 70-Minuten-Pflichtweg und zu den Zusatzaufgaben. Die Nummern entsprechen genau dem Leitprogramm. Messwerte hängen vom ZIP-Programm, den Dateinamen und den Textzeilen ab. Bewerte deshalb den Rechenweg und die Erklärung, nicht die Übereinstimmung mit einer ZIP-Referenzgrösse.

Aktuell geprüfte Ausgangsdateien

Datei	Grösse in Bytes
Bild(1).jpg	716’894
BildBMP(1).bmp	25’247’934
Alarm05(1).wav	10’406’738

Die ältere Grundlage nennt für das JPEG 7’706’104 Bytes. Dieser Wert passt nicht zur aktuell bereitgestellten Datei und wird hier nicht übernommen.

Lösung 1 Arbeitsordner vorbereiten

1.1 Dateigrösse

Die JPEG-Datei ist 716’894 Bytes gross. In Windows kann daneben eine gerundete Angabe in KB oder MB stehen. Für Rechnungen ist die Byte-Zahl in Klammern geeignet. Grösse auf Datenträger beschreibt den belegten Platz in Speicherblöcken und kann abweichen. Sie wird hier nicht verwendet.

Erwarteter Zustand: Im selbst erstellten Ordner liegen drei unabhängige Kopien. Die Endungen .jpg, .bmp und .wav sind sichtbar. Kopieren mit Strg + C und Strg + V erhält den Inhalt. Das Umbenennen allein komprimiert keine Datei.

Typische Schwierigkeiten

Eine andere Ausgangsgrösse kann auf eine andere Datei hinweisen. Ein Download oder erneuter JPEG-Export kann den Inhalt verändern. Die Lehrperson soll zuerst prüfen, welche Datei tatsächlich vorliegt. Unterschiedliche Zahlen in Grösse und Grösse auf Datenträger sind dagegen normal.

Lösung 2 Wiederholungen untersuchen

2.1 Grösse der Textdatei

Ein Wort KOMPRESSION enthält 11 Zeichen. Vier Wörter und drei Leerzeichen ergeben 47 Zeichen. Bei diesen Zeichen benötigt UTF-8 ein Byte pro Zeichen. Ein Windows-Zeilenumbruch benötigt meist zwei weitere Bytes. Eine vollständige Zeile benötigt dann 49 Bytes.

Sechs Verdopplungen ergeben 1 → 2 → 4 → 8 → 16 → 32 → 64 Zeilen. Mit einem abschliessenden Windows-Zeilenumbruch pro Zeile sind das 64 × 49 = 3’136 Bytes. Ohne letzten Zeilenumbruch sind es 3’134 Bytes. Andere Zeilenumbrüche, eine UTF-8-Markierung oder eine andere Zeilenzahl verändern die Grösse. Akzeptiere die tatsächlich korrekt abgelesene Zahl.

2.2 Muster

Das Wort KOMPRESSION kommt in jeder Zeile viermal vor. Ausserdem wird dieselbe ganze Zeile immer wiederholt. Ein Kompressionsverfahren kann auf bereits vorkommende Muster verweisen, statt alles erneut auszuschreiben. Diese Wiederholungen heissen Redundanz.

2.3 Zeichenfolgen

AAAAAAAAAAAAAAAAAAAA besitzt mehr sichtbare Wiederholungen: A kommt zwanzigmal hintereinander vor. Mit einer vereinbarten Kurzform lässt sich etwa „20-mal A“ speichern. Die zweite Folge hat kein ähnlich auffälliges Muster. Das ist eine Vermutung über die Daten; ein konkretes Verfahren und seine Verwaltungsdaten bestimmen die wirkliche Dateigrösse.

Lösung 3 ZIP Messwerte

Diese Referenzarchive wurden mit ZIP und Deflate Stufe 6 erzeugt. Im Archiv stehen die Dateinamen ohne (1). Windows kann andere ZIP-Grössen erzeugen. Vergleiche jeweils die gesamte Archivgrösse mit der Ausgangsdatei.

Ausgangsdatei	Vorher Bytes	ZIP Bytes	Einsparung
Bild(1).jpg	716’894	716’737	0,0 %
BildBMP(1).bmp	25’247’934	19’855’375	21,4 %
Alarm05(1).wav	10’406’738	9’690’890	6,9 %

Muster für die Textdatei: 64 identische Zeilen mit Windows-Zeilenumbrüchen ergeben 3’136 Bytes. Ein Referenz-ZIP mit dem Eintrag Wiederholung.txt hat 177 Bytes. Diese Werte dienen nur als Rechenbeispiel; die Lernenden verwenden ihre eigenen Werte.

3.1 Vermutung

Meist wird Wiederholung.txt prozentual am stärksten verkleinert. Sie enthält besonders viele wiederkehrende Wörter und ganze Zeilen. Erst die Messung bestätigt die Vermutung. BMP kann auch deutlich kleiner werden. JPEG ist bereits komprimiert. Bei WAV hängt die Einsparung vom Inhalt und der gespeicherten Audioart ab.

Ein Archiv pro Ausgangsdatei ermöglicht eine faire Zuordnung der Einsparung. Ein gemeinsames Archiv würde nur die Einsparung des ganzen Pakets zeigen. Dateinamen und Archiv-Verwaltungsdaten gehören zur hier gemessenen ZIP-Grösse.

Lösung 4 Entpacken und vergleichen

4.1 Vor der Änderung

Ja. Unmittelbar nach dem fehlerfreien Entpacken stimmen die ursprünglichen Bytes überein. Damit sind auch der Text und die Dateigrösse gleich. Wenn die Lernenden die entpackte Datei bereits gespeichert oder geändert haben, sollen sie für diese Kontrolle nochmals in einen neuen Ordner entpacken.

Wichtig für die Beurteilung: Gleiche sichtbare Wörter und gleiche Grösse allein beweisen keine Byte-Gleichheit. Beispielsweise können zwei unterschiedlich geschriebene Wörter dieselbe Länge haben. Die einfache Kontrolle macht das Prinzip anschaulich. Ein exakter Nachweis würde einen Byte-Vergleich benötigen.

4.2 Nach der Änderung

Nein. Die entpackte Datei ist eine eigene Kopie. Das Original im Arbeitsordner und der Inhalt des ZIP-Archivs bleiben unverändert. In der entpackten Kopie steht zuerst TEST, im Original weiterhin KOMPRESSION. TEST ist kürzer; deshalb wird die entpackte Textdatei bei unveränderter Codierung kleiner.

4.3 Kompressionsart

ZIP ist verlustfrei. Das Verfahren speichert die ursprünglichen Informationen platzsparend. Beim Entpacken werden sie vollständig wiederhergestellt. Es dürfen dabei keine Zeichen, Toninformationen oder Bildbytes fehlen. Verlustbehaftete Kompression entfernt dagegen Informationen, die nachher nicht exakt wiederhergestellt werden können.

Lösung 5 Rechnungen und Erklärungen

Musterrechnung für die Textdatei

Für D_vor = 3’136 Bytes und D_nach = 177 Bytes:
K = 3’136 ÷ 177 = 17,7
R = 177 ÷ 3’136 × 100 % = 5,6 %
E = 100 % − 5,6 % = 94,4 %
Die Ausgangsdatei ist ungefähr 17,7-mal so gross wie das Archiv. Rund 5,6 % bleiben übrig; rund 94,4 % werden eingespart. Rechne intern ungerundet und runde erst die Endwerte.

Einsparungen der Beispieldateien

JPEG: E = 100 − (716’737 ÷ 716’894 × 100) ≈ 0,0 %. Ungerundet sind es etwa 0,022 %; 157 Bytes wurden eingespart.
BMP: E = 100 − (19’855’375 ÷ 25’247’934 × 100) ≈ 21,4 %.
WAV: E = 100 − (9’690’890 ÷ 10’406’738 × 100) ≈ 6,9 %.

5.1 Stärkste Einsparung

Im Referenzvergleich gewinnt der wiederholte Text. ZIP kann die ständig gleichen Abschnitte sehr kurz darstellen. Entscheidend ist die prozentuale Einsparung, nicht nur die Zahl eingesparter Bytes. Ein grosses BMP kann absolut mehr Bytes einsparen und prozentual trotzdem schlechter abschneiden.

5.2 JPEG

JPEG hat die Bildinformationen bereits komprimiert. ZIP findet deshalb meist nur noch wenig weitere nutzbare Wiederholungen. ZIP verändert dabei die JPEG-Datei nicht verlustbehaftet und verschlechtert ihre Bildqualität nicht.

5.3 Expansion

R = 1’050 ÷ 1’000 × 100 % = 105 %. E = 100 % − 105 % = −5 %. Das Archiv ist 5 % grösser geworden. Es wurde kein Platz eingespart. Mögliche Gründe sind Verwaltungsdaten des Archivs oder Daten, die sich kaum weiter komprimieren lassen. K = 1’000 ÷ 1’050 ≈ 0,95; es liegt unter 1.

Lösung 6 Auswahl und Abschlusskontrolle

6.1 Python Projekt

Verlustfrei, zum Beispiel ZIP. Jeder Buchstabe und jedes Zeichen muss erhalten bleiben. Ein fehlender Doppelpunkt oder ein veränderter Name kann das Programm unbrauchbar machen.

6.2 Foto für eine Webseite

Verlustbehaftet, zum Beispiel JPEG. Eine kleinere Datei lässt sich bei gleicher Verbindung schneller laden. Eine leichte Qualitätsänderung ist hier erlaubt. Das hochwertige Original wird zusätzlich behalten. Eine technisch begründete andere geeignete Wahl ist ebenfalls zulässig.

6.3 Transparentes Logo

Verlustfrei, zum Beispiel PNG. Scharfe Kanten und Schrift sollen erhalten bleiben. PNG unterstützt einen transparenten Hintergrund. JPEG unterstützt diese Transparenz nicht.

6.4 Hauptziel

Daten sollen weniger Speicherplatz benötigen. So lassen sie sich häufig mit weniger Datenverkehr übertragen.

6.5 Kennzahlen

K = 100 ÷ 40 = 2,5. R = 40 ÷ 100 × 100 % = 40 %. E = 100 % − 40 % = 60 %. Der Faktor hat keine Einheit. Restgrösse und Einsparung dürfen nicht vertauscht werden.

6.6 Schutz

Nein. Eine normale ZIP-Datei kann jede Person mit einem passenden Programm öffnen. Kompression spart Platz. Für Schutz vor unbefugtem Lesen braucht es zusätzlich eine geeignete Verschlüsselung.

6.7 Verlorene JPEG Informationen

Nein. ZIP erhält die vorhandenen JPEG-Bytes. Bildinformationen, die bereits beim früheren verlustbehafteten JPEG-Speichern entfernt wurden, sind darin nicht mehr enthalten. ZIP kann sie nicht zurückbringen.

Lösungen zu den Zusatzaufgaben

Zusatz A Kleine Datei

Eine Datei mit nur A benötigt bei UTF-8 ohne zusätzliche Markierung und ohne Zeilenumbruch 1 Byte. Mit anderen Speichereinstellungen kann sie etwas grösser sein. Ein ZIP mit dem Eintrag Klein.txt kann in einer Deflate-Referenz beispielsweise 119 Bytes benötigen. Dann gilt E = 100 − (119 ÷ 1 × 100) = −11’800 %. Das Archiv ist um 118 Bytes grösser. Der grosse Prozentwert entsteht durch die winzige Ausgangsgrösse.

Das Archiv braucht Angaben zum Dateinamen, zum Inhalt und zum Entpacken. Diese Verwaltungsdaten sind bei einer so kleinen Datei viel grösser als eine mögliche Einsparung. Akzeptiere andere gemessene ZIP-Grössen bei passender Rechnung und Erklärung.

Zusatz B Übertragungszeit

K = 800 ÷ 200 = 4. R = 200 ÷ 800 × 100 % = 25 %. E = 75 %. Vorher: 800 MB ÷ 10 MB pro Sekunde = 80 Sekunden. Nachher: 200 MB ÷ 10 MB pro Sekunde = 20 Sekunden. Einsparung: 80 − 20 = 60 Sekunden. In der Praxis kostet auch das Komprimieren und Entpacken Zeit; diese Zeiten wurden in der Aufgabe ausdrücklich ausgeklammert.

Zusatz C Kurzform

AAAAAAAAAABBBBCC wird zu 10A4B2C. Es sind zehn A, vier B und zwei C. 2A3B1C wird zu AABBBC. Die vereinbarte Regel erlaubt es, die ursprüngliche Folge wieder genau aufzubauen: Die Darstellung ist verlustfrei.

ABABAB wird mit dieser einfachen Regel zu 1A1B1A1B1A1B. Die Kurzform besitzt zwölf statt sechs Zeichen. Die gleichen Buchstaben stehen nicht direkt hintereinander. Diese Lauflängenregel hilft deshalb nicht. Andere Verfahren können das wiederholte Muster AB trotzdem nutzen. Daraus folgt nicht, dass ABABAB grundsätzlich schlecht komprimierbar wäre.